Social mood matcher

Team Id: 69a5aac8cdc547b196fdca1e

Team members: 5

Team leader: V. BHUVANESHWARI

Team member: U. NIVETHA

Team member: T.S ROHINI

Team Member: K. DEEPASRI

Team Member: S. SARANYA

Project Description

Creating engaging social media content with appropriate captions and trending hashtags is time-consuming for content creators, marketers, and social media managers. Users often spend significant time crafting the perfect caption, researching relevant hashtags, and ensuring content fits within platform character limits across different social media platforms.

Social Mood Matcher addresses this challenge through a GenAI-powered social media content generation platform that delivers personalized, sentiment-aware captions and trending hashtags optimized for different platforms. Using advanced image analysis with BLIP for image captioning, DistilBERT for sentiment analysis, and optionally Google Gemini 2.5 Flash for enhanced quality, the platform analyzes uploaded images to detect mood and visual context.

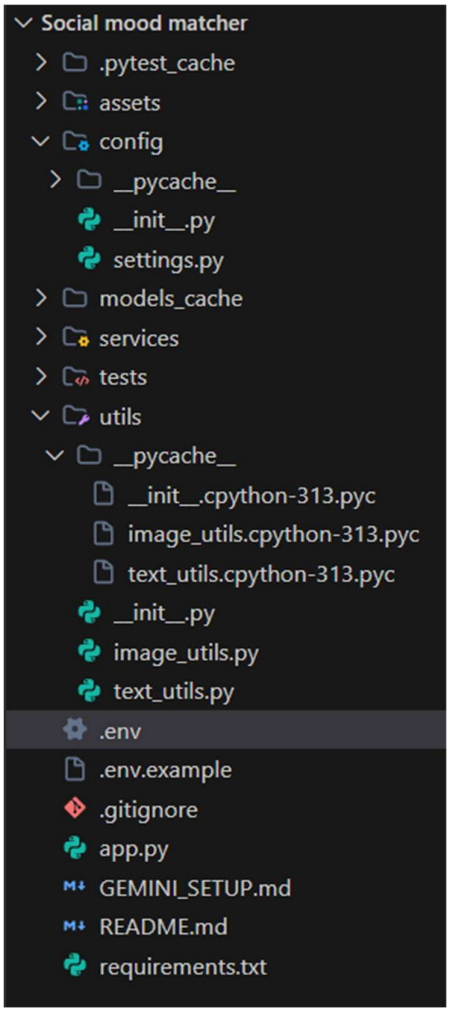
The platform helps content creators generate engaging captions in multiple styles (casual, aesthetic, professional, playful), provides curated 2024 trending hashtags organized by category and sentiment, and ensures all content fits within platform-specific character limits for Twitter/X (280 chars), Instagram (2,200 chars), and Facebook (63,206 chars). Social Mood Matcher transforms social media content creation into an intelligent, efficient process through its smart sentiment detection, adaptive caption generation, and multi-platform export capabilities.

MILESTONE 1: Environment Setup & Configuration

In this foundational stage, the objective is to establish the development environment and configure all necessary tools, libraries, and AI services required for Social Mood Matcher

Activity 1.1: Project Structure Setup

Create the project directory structure with proper organization for services, utilities, configuration, and tests



Activity 1.2: Install Dependencies

Install all required Python packages using pip. The requirements.txt includes Streamlet,Transformers, PyTorch, Pillow, and optional Gemini API support

```
import streamlit as st
from PIL import Image
import sys
from pathlib import Path
```

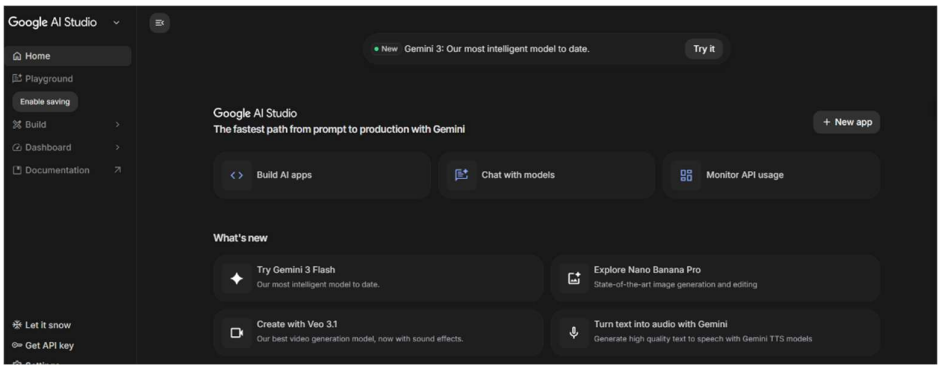
Activity 1.3: Configure Environment Variables

Set up the .env file with necessary configurations. This includes optional Gemini API key,model cache directory, and debug settings.

```
# Social Mood Matcher - Environment Configuration
# This file contains your actual API keys and settings

# Debug Settings
DEBUG=False
LOG_LEVEL=INFO

# Model Cache Directory
MODEL_CACHE_DIR=./models_cache

# Google Gemini API Key
GEMINI_API_KEY=Your API key

# AI Model Selection (local or gemini)
AI_MODEL=gemini

# Device Configuration (cpu or cuda)
# DEVICE=cpu
```

Activity 1.4: Optional Gemini Setup

1. Visit Google AI Studio: https://makersuite.google.com/app/apikey



2. Sign in with Google account

3. Click 'Create API Key' and copy the generated key

4. Add key to .env file: GEMINI_API_KEY=your_key_here

```
GEMINI_API_KEY=your_api_key_here
SECRET_KEY=your_secret_key_here
```

5. Set AI_MODEL=gemini in .env to enable enhanced mode

6. Install google-generativeai package (included in requirements.txt)

MILESTONE 2: Core Services Development

This milestone focuses on building the core AI-powered services that form the backbone of Social Mood Matcher. Each service is designed as an independent, testable module with clear Responsibilities.

Activity 2.1: Image Sentiment Detector Service

Create the image_sentiment.py service that uses BLIP for image captioning and Distil BERT

for sentiment analysis. This service is the first step in the content generation pipeline.

Key Features:

• Load and cache BLIP image captioning model

• Load and cache DistilBERT sentiment analysis model

• Process images to extract visual descriptions

• Analyze sentiment from image captions and visual cues

• Classify into 10 mood categories (happy, cozy, aesthetic, calm, etc.)

• Detect image categories (food, travel, nature, lifestyle)

• Return confidence scores for all detected sentiments

Activity 2.2: Caption Generator Service

Develop the caption_generator.py service that creates engaging captions based on sentiment, style preferences, and image context.

Supported Styles:

Style	Characteristics	Emoji Usage
Casual	Friendly, relaxed tone	Moderate (2-3)
Aesthetic	Poetic, artistic language	Minimal (0-1)
Professional	Polished, informative	None
Playful	Fun, energetic tone	High (3-5)

Activity 2.3: Hashtag Engine Service

Build the hashtag_engine.py service that provides curated, trending hashtags organized bycategory and sentiment. Includes 2024's most popular hashtags for maximum engagement.

Hashtag Categories:

• Food: #FoodPhotography, #Foodie, #InstaFood, #FoodLover, #Delicious

•Travel: #TravelPhotography, #Wanderlust, #TravelGram, #Explore, #Adventure

•Nature: #NaturePhotography,#NatureLover, #Landscape, #Scenic, #OutdoorLife

• Lifestyle: #LifestylePhotography, #DailyLife, #Moments, #Inspiration, #Aesthetic

Activity 2.4: Character Limiter Service

Implement the character_limiter.py service that ensures all generated content fits within platform-specific character limits while maintaining message integrity

Platform	Character Limit	Truncation Strategy
Twitter/X	280	Prioritize caption, truncate hashtags
Instagram	2,200	Rarely needed, full content usually fits

Facebook	63,206	Almost never needed

Activity 2.5: Optional Gemini Service Integration

Create the gemini_service.py for users who want enhanced AI capabilities. This service provides superior caption quality, multiple variants, and visual intelligence features.

- Enhanced sentiment detection with higher accuracy
- Multiple caption variants (aesthetic, punchy, descriptive)
- Visual intelligence: dominant colors, detected objects
- Composition tips for better photography
- More creative and contextually relevant captions

MILESTONE 3: Backend Implementation

This milestone integrates all services into a cohesive backend architecture within the Streamlit application. Focus on proper session state management, error handling, and efficient caching.

```python
class GeminiVisionAnalyzer:
    """Analyzes images using Google Gemini Vision API."""

    def __init__(self, api_key: Optional[str] = None):
        """
        Initialize Gemini API client.

        Args:
            api_key: Google Gemini API key (uses env var if not provided)
        """
        self.api_key = api_key or os.getenv("GEMINI_API_KEY") or API_KEYS.get("gemini")

        if not self.api_key:
            raise ValueError(
                "Gemini API key not found. Please set GEMINI_API_KEY in .env file or environment variables."
            )

        # Configure Gemini
        genai.configure(api_key=self.api_key)

        # Initialize model
        self.model = genai.GenerativeModel('gemini-1.5-flash')

        print("Gemini API initialized successfully")

    def analyze_image_sentiment(self, image: Image.Image) -> Dict:
        """
        Analyze image sentiment using Gemini Vision.

        Args:
            image: PIL Image object

        Returns:
            Dictionary with sentiment analysis results
```

Activity 3.1: Session State Management

Implement proper Streamlit session state initialization to maintain user selections, generated content, and application state across reruns.

- **generated_caption**: Current generated caption
- **generated_hashtags**: Current hashtag list
- **sentiment_result**: Detected sentiment and metadata
- **caption_variants**: Multiple caption options (Gemini mode)
- **visual_analysis**: Visual intelligence data (Gemini mode)
- **history**: List of previous generations
- **use_gemini**: Toggle for Gemini vs local models

• **gemini_analyzer**: Cached Gemini service instance

Activity 3.2: Service Initialization and Caching

Use Streamlit's @st.cache_resource decorator to load AI models once and reuse across sessions. This significantly improves performance by avoiding repeated model loading.

• ImageSentimentDetector: BLIP and DistilBERT models (~2GB)

• CaptionGenerator: Template-based generation system

• HashtagEngine: Curated hashtag database

• CharacterLimiter: Platform rules and truncation logic

• GeminiAnalyzer (Optional): Gemini API client

Activity 3.3: Error Handling and Validation

Implement comprehensive error handling for file uploads, model inference, and API calls. Provide clear error messages and fallback mechanisms.

1.Invalid file format: Show supported formats (JPG, PNG, WEBP)

1. File size too large: Display max upload size (10MB)
2. Model loading failure: Show retry option or use cached models
3. Gemini API error: Automatically fall back to local models
4. Empty generation: Retry with adjusted parameters

Activity 3.4: Image and Text Processing Utilities

Implement utility functions in utils/ directory for common image and text operations used throughout the application.

image_utils.py: validate_image(),resize_image(),preprocess_for_model(), create_thumbnail()

text_utils.py: clean_text(), format_hashtags(), truncate_text(), smart_truncate_with_hashtags()

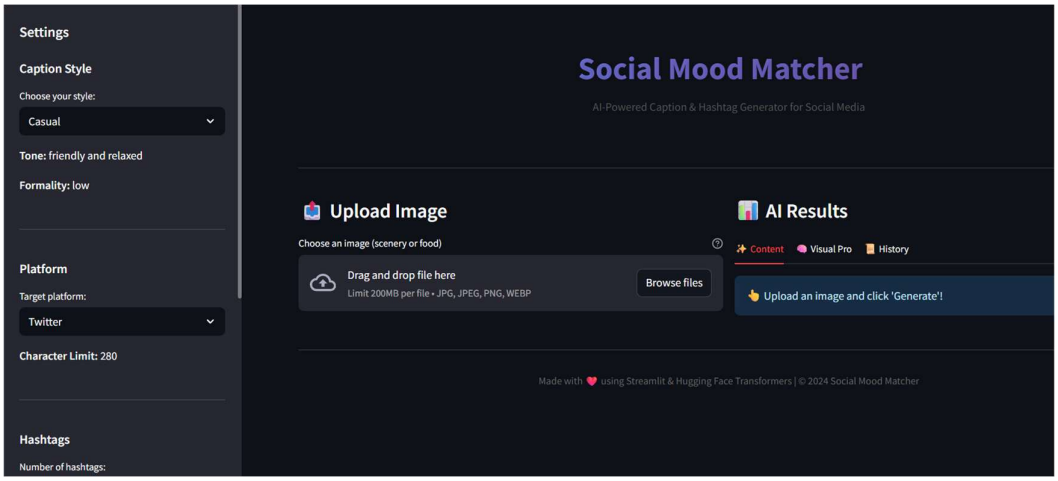
MILESTONE 4: UI Development

Create an intuitive, visually appealing Streamlit interface with responsive design, clear navigation, and real-time feedback during content generation.

Activity 4.1: Main Layout Design

Design a two-column layout with image upload on the left and results display on the right. Include custom CSS for enhanced visual appeal.

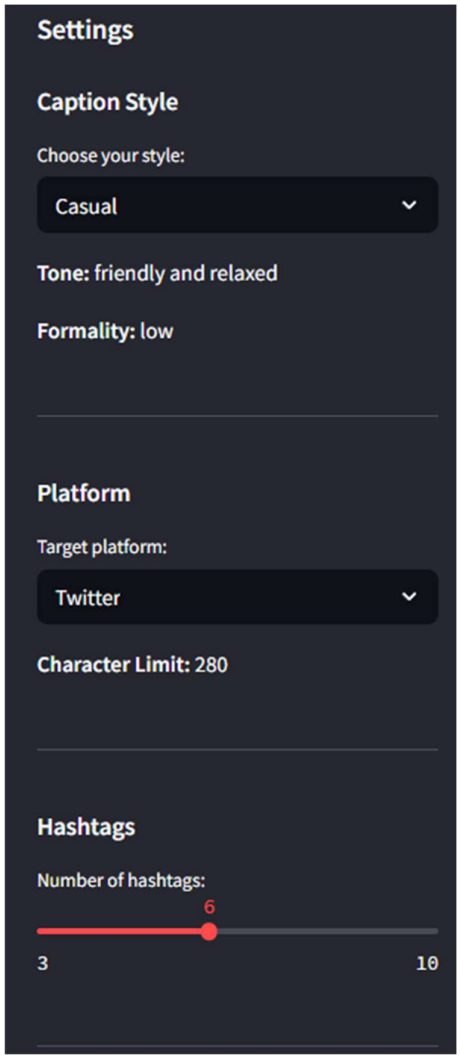
- Header with gradient title and subtitle
- Sidebar for settings (style, platform, hashtag count, Gemini toggle)
- Left column: Image upload with preview
- Right column: Tabbed results (Content, Visual Pro, History)
- Custom color scheme matching brand (purple gradient)
- Responsive design for mobile and desktop



Activity 4.2: Sidebar Controls

Implement sidebar with user controls for customizing generation parameters.

- **Caption Style:** Select from Casual, Aesthetic, Professional, Playful
- **Platform:** Choose Twitter/X, Instagram, or Facebook
- **Number of Hashtags:** Slider from 3 to 15
- **Use Gemini API:** Toggle for enhanced vs local mode
- **About Section:** App description and features



Activity 4.3: Results Display

Create tabbed interface for displaying different types of results:

Content Tab: Sentiment badge, caption variants selector (Gemini), generated caption, hashtags, character count

Visual Pro Tab: Visual intelligence (Gemini only): dominant colors, detected objects, composition tips

• **History Tab:** Previous 5 generations with sentiment and timestamp

activity 4.4: Export Functionality

Add download button for exporting generated content as a text file. Content includes caption and hashtags formatted for easy copy-paste.

MILESTONE 5: Testing & Optimization

Comprehensive testing ensures reliability, accuracy, and performance. This milestone includes unit tests, integration tests, and performance optimization.

Activity 5.1: Unit Testing

Test individual components using pytest framework. Each service should have comprehensive test coverage.

- **TestImageProcessor:** Image validation, resizing, preprocessing
- **TestTextProcessor:** Text cleaning, hashtag formatting, truncation
- **TestCaptionGenerator:** Caption generation for all styles and sentiments
- **TestHashtagEngine:** Hashtag retrieval, filtering, priority-based selection
- **TestCharacterLimiter:** Platform limits, smart truncation, edge cases

Activity 5.2: Integration Testing

Test the complete pipeline from image upload to final content generation.

- Upload image → Sentiment detection → Caption generation → Hashtag retrieval → Character limiting
- Test with multiple image types: food, travel, nature, lifestyle
- Verify all caption styles produce appropriate outputs
- Validate character limits for all platforms
- Test Gemini fallback when API fails

Activity 5.3: Performance Optimization

Optimize application performance for faster response times and better user experience.

- Use @st.cache_resource for model loading (prevents reloading)
- Implement image resizing before model inference (faster processing)
- Add loading spinners during AI operations (better UX)
- Cache Gemini API client to avoid repeated initialization
- Optimize session state updates to minimize reruns

Expected Performance Metrics:

Operation	First Run	Subsequent Runs
Model Loading	30-60 seconds	Instant (cached)
Image Analysis	3-5 seconds	2-3 seconds
Caption Generation	1-2 seconds	1-2 seconds
Gemini API Call	2-4 seconds	2-4 seconds
Total (Local Mode)	~35-70 sec first	~5-8 sec
Total (Gemini Mode)	~5-10 sec	~4-6 sec

Deployment Guide

Social Mood Matcher can be deployed on multiple platforms. This section covers Streamlit Cloud, and cloud platforms (AWS/GCP/Azure).

Option 1: Streamlit Cloud (Recommended)

1. Push code to GitHub repository
2. Visit https://share.streamlit.io and sign in
3. Click 'New app' and select your repository
4. Configure main file path (app.py) and Python version (3.10)
5. Add secrets: Go to Advanced Settings → Secrets
6. Deploy and wait for build completion (~5 minutes)

Option 2: Cloud Platforms (AWS/GCP/Azure)

Deploy on cloud platforms for production-grade scalability and reliability. Use managed container services like AWS ECS, Google Cloud Run, or Azure Container Instances.

Troubleshooting Guide

Issue: Models not loading

- Check internet connection for first-time download
- Verify MODEL_CACHE_DIR has write permissions
- Ensure sufficient disk space (~2GB)
- Try clearing cache: rm -rf models_cache/

Issue: Gemini API errors

- Verify API key is correct in .env file
- Check API quota at Google AI Studio
- Ensure google-generativeai package is installed
- Application automatically falls back to local models

Issue: Session state issues

- Clear browser cache and reload page
- Check for proper session state initialization
- Verify all session variables are defined

Issue: Character limit not working

- Verify platform selection matches expected limit

• Check smart_truncate_with_hashtags() logic

• Test with different caption/hashtag combinations

Issue: Image upload failures

• Check file format (JPG, PNG, WEBP only)

• Verify file size is under 10MB

• Ensure image is not corrupted

• Try different image or resize before upload

Conclusion

Social Mood Matcher revolutionizes social media content creation by delivering intelligent, AI-driven captions and hashtags tailored to image sentiment and user preferences. By combining BLIP for image understanding, DistilBERT for sentiment analysis, and optionally Gemini 2.5 Flash for enhanced quality, the system processes visual content and generates engaging, platform-optimized social media posts.

From modular service architecture to intelligent content generation and responsive UI design, Social Mood Matcher has been built with focus on usability, reliability, and content creator needs. The Streamlit interface ensures intuitive interaction, while the robust backend handles model inference, error scenarios, and performance optimization.

Each component—from sentiment detection to character limiting—is designed to save content creators time while maintaining high-quality standards. Whether creating a quick Instagram post or crafting the perfect Twitter caption, Social Mood Matcher acts as an intelligent content assistant, automating the creative process while preserving authenticity and engagement potential.

Future Enhancements

- Support for video content analysis
- Multi-language caption generation
- A/B testing for caption variants
- Social media analytics integration
- Batch processing for multiple images
- Custom hashtag training based on user preferences
- Direct posting to social media platforms
- Brand voice customization